NERIUM

INFRASTRUCTURE FOR THE AI AGENT ECONOMY

Product Requirements Document

Version 2.0

Cerebral Valley + Anthropic Hackathon
Built with Opus 4.7
April 2026

Authored by Ghaisan Khoirul Badruzaman
Solo founder

CLASSIFIED

This document contains confidential and proprietary information.

Table of Contents

1. Executive Summary

2. Problems Solved

2.1 The Individual Gap: Idea to Product

2.2 The Systemic Gap: Fragmented Agent Economy

2.3 The Core Insight

3. The Solution: NERIUM

4. Five Platform Pillars

4.5 Hackathon Submission Status (April 2026)

4.5.1 Build Context

4.5.2 Recursive Automation Thesis

4.5.3 What Ships at Submission

4.5.4 Theatrical Demo Plus BYOK Pattern

4.5.5 Multi-Vendor Showcase, Anthropic-Only Invocation

4.5.6 Deployment State

4.5.7 Honest Disclaimers, Summarized

5. Pillar 1: Builder (Gamified Product Construction)

5.1 Overview

5.2 The JRPG World Wrapper

5.3 The Building Metaphor

5.4 The Sekuri Agent Tier System

5.5 The Multi-Vendor Selection Moment

5.6 The Agent Cascade

5.7 The Six-Phase Automation Pipeline

5.8 Honest-Claim Disclaimers for Builder

6. Pillar 2: Marketplace (Open Agent Economy)

6.1 Overview

6.2 Open Platform Philosophy

6.3 Living Templates

CLASSIFIED

CONFIDENTIAL / PROPRIETARY

6.4 The Restaurant Automation Example

6.5 Rating and Review System

6.6 Competitive Moat

7. Pillar 3: Banking (Financial Infrastructure)

7.1 Overview

7.2 The Future Context

7.3 Core Functions

7.4 Honest-Claim Disclaimer for Banking

7.5 Strategic Role

8. Pillar 4: Registry (Identity and Trust Layer)

8.1 Overview

8.2 The Problem

8.3 Core Functions

8.4 The DNS Analogy

9. Pillar 5: Protocol (Interoperability Standard)

9.1 Overview

9.2 The Fundamental Problem

9.3 Approach: Translation Layer, Not Single Language

9.4 The Eight-Vendor Surface

9.5 The Sekuri Large Template as Concrete Example

9.6 Industry Context

9.7 Honest-Claim Disclaimer for Protocol

9.8 Long-Term Vision

10. User Journeys

10.1 Journey A: Builder (Building from Zero)

10.2 Journey B: Marketplace (Buying and Customizing)

10.3 Journey C: Banking and Registry (Live Agent Operations)

10.4 Journey D: The Creator Path (Recursive)

11. Business Model and Monetization

CONFIDENTIAL / PROPRIETARY

11.1 AI Model Usage Fees (Builder)

11.2 Platform Subscription

CLASSIFIED

11.3 Marketplace Transaction Fees
11.4 Banking Processing Fees
11.5 Registry Premium Services
11.6 Protocol Licensing (Long-Term)
11.7 Flywheel Revenue
12. Market Size and Opportunity
12.1 The Global AI Agent Market
12.2 Adjacent Markets
12.3 Timing
13. Target Audience
13.1 Builder Audience
13.2 Marketplace Audience
13.3 Banking, Registry, and Protocol Audience
14. Competitive Landscape and Differentiation
14.1 Direct Competitors (Builder)
14.2 NERIUM Builder Differentiation
14.3 Competitors in the Marketplace and Infrastructure Space
15. Risks and Considerations
15.1 Quality of AI-Generated Code
15.2 Scope Management
15.3 API Cost Uncertainty
15.4 User Expectations
15.5 Cold Start Problem
15.6 Regulation and Compliance
15.7 Security and Trust
15.8 Execution Complexity
15.9 Solo Founder Risk
16. Success Metrics
16.1 Engagement and Adoption
16.2 Product Quality
16.3 Economic Viability

CLASSIFIED
CONFIDENTIAL / PROPRIETARY

16.4 Submission-Window Metrics

17. Future Vision

17.1 The 2027 Milestone: Idea to App Store

17.2 The 2029 Vision: Civilization-Scale Infrastructure

17.3 Multiplayer Mode

17.4 Building District

17.5 Maintenance Mode

17.6 Building Upgrades

17.7 Agentic Commerce

17.8 Personal Agent Ecosystem

17.9 Industry Standard

17.10 The Founder's Note

Closing

CLASSIFIED

CONFIDENTIAL / PROPRIETARY

NERIUM

Infrastructure for the AI Agent Economy

Product Requirements Document, Version 2.0

April 2026

Authored by Ghaisan Khoirul Badruzaman, solo founder

Status: Hackathon submission baseline (Cerebral Valley plus Anthropic, "Built with Opus 4.7")

> *"NERIUM will make people addicted to creating prompts. We stop worrying about code. We start thinking only in ideas. You can even ask NERIUM to generate ideas, and execute them."*
>
> FOUNDER THESIS, SESSION 2 VOICEOVER, APRIL 2026

CONFIDENTIAL / PROPRIETARY

1. Executive Summary

NERIUM is infrastructure for the AI agent economy. Five integrated pillars (Marketplace, Builder, Banking, Registry, Protocol) sit beneath every agent that gets built, sold, executed, identified, and routed across model vendors. The platform is not another AI assistant. It is the layer above models, the layer below applications, and the connective tissue between the two.

The thesis is simple. AI agents are proliferating faster than the infrastructure that should support them. Skills get traded on Twitter. Prompts get pasted into Discord. MCP servers sit forgotten on GitHub. Plugins live scattered across eight different vendor stores. Creators ship work nobody can find, and buyers hunt agents by direct-messaging strangers about payment. The supply side is fragmented, the demand side is searchless, the payment rails do not exist, the trust layer is absent, and the communication protocols are vendor-locked. Every problem in this list is solvable with infrastructure, and infrastructure is what NERIUM builds.

NERIUM was constructed during the Cerebral Valley plus Anthropic "Built with Opus 4.7" hackathon in April 2026. The submission codebase was authored by 54-plus specialist Claude Code agents on Opus 4.7, orchestrated through formal handoff documents (V1 through V6). The recursive nature is the product's central frame: NERIUM built itself by running, one last time manually, the exact multi-agent workflow that NERIUM Builder is designed to automate. Every handoff, every parallel agent group, every contract validation, every QA pass that the submission required is precisely what Builder collapses into a single conversational interface.

The global AI agent market is projected to grow from USD 5.1 billion in 2024 to between USD 47 billion and USD 52 billion by 2030, at a compound annual growth rate between 41 and 46 percent. NERIUM does not compete in this market. NERIUM provides the infrastructure on which this market operates. The platform's positioning is analogous to AWS for compute, Stripe for payments, DNS for naming, and HTTP for transport. Each of those layers won by being the substrate, not the application.

By 2027, NERIUM intends to ship from idea to App Store production deployment in a single conversation. By 2029, the thesis is civilization-scale: software development is no longer a profession bottleneck, every business runs on agents, every agent runs on NERIUM, and ideas (not codebases) create wealth. The hackathon submission is the proof of concept for that trajectory at the scale a solo founder can ship in a week.

CONFIDENTIAL / PROPRIETARY

2. Problems Solved

2.1 The Individual Gap: Idea to Product

Thousands of would-be founders carry ideas they believe in. Most of them will never build anything. The reason is rarely the idea. The reason is the distance between holding an idea and shipping a product, and that distance is built out of four hard barriers.

The knowledge barrier. Turning an idea into a product requires fluency in product management, software architecture, user experience design, development workflows, testing, deployment, and a long tail of adjacent disciplines. Most people with strong ideas do not carry this knowledge, and acquiring it takes years.

The cost barrier. Hiring a development team to build even a modest minimum viable product runs into the tens of thousands of dollars. Production-grade products move that figure into six digits. For most individuals and most small businesses, this places software development entirely out of reach.

The complexity barrier. Even for those with technical fluency, modern software development means coordinating dozens of moving parts: frontend, backend, database, API, authentication, hosting, continuous integration, security, observability, billing, compliance. Orchestrating all of it is a full-time job in itself, and that job exists before any actual product features get written.

The motivation barrier. Building a product is a long, often solitary process. Without visible progress, fast feedback, and a sense of momentum, most builders abandon their work before reaching anything shippable. The process feels like running on a treadmill in the dark, and most people, reasonably, get off.

NERIUM Builder addresses all four. The knowledge barrier collapses because Builder's agent roster carries the expertise. The cost barrier collapses because the marginal cost of running 14 specialist agents in parallel is dramatically lower than hiring 14 specialists. The complexity barrier collapses because the orchestration is automated. And the motivation barrier collapses because the experience is structured as a game, with visible progress, narrative anchors, and persistent momentum.

2.2 The Systemic Gap: Fragmented Agent Economy

The systemic problem is larger and more structural. The global AI agent economy is fragmenting along five distinct axes, and no existing platform addresses any of them at infrastructure scale.

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

No unified marketplace. Agents are being authored everywhere: GitHub repositories, vibe-coding platforms, indie developer Twitter, enterprise vendor stores, Discord servers, Hugging Face Spaces, vibe-coding tool galleries. There is no single, standardized place to discover, evaluate, purchase, and sell agents. Supply is scattered. Demand is searchless. The .skills traded on Twitter and the MCP servers buried in unstarred GitHub repositories are concrete artifacts of this fragmentation.

No payment infrastructure. When agents go live and consume resources continuously through APIs, IoT pipelines, or cloud services, no financial system is purpose-built to handle their billing. Stripe handles SaaS subscriptions. Stripe Connect handles marketplace payouts. Neither is shaped for per-query agent billing, agent-to-agent settlement, escrow between autonomous parties, or revenue splits between creators, platforms, and execution vendors. The agent economy currently runs on adapters of adapters.

No trust system. Anyone can publish an agent, and no standardized mechanism exists to verify identity, confirm capabilities, audit security, or surface track record. The current state of the agent economy is unregulated open territory. For consumer use this is tolerable. For enterprise adoption it is a non-starter.

No universal communication standard. Each AI vendor has its own preferred prompting style, output format, and reasoning conventions. Claude works best with XML tags and CLAUDE.md files. Gemini has its own conventions. OpenAI's models follow yet another set of patterns. When agents from different models need to collaborate, no translation layer handles vendor-specific optimization automatically. The Model Context Protocol (MCP) and Agent-to-Agent Protocol (A2A) standards exist and are valuable, but they handle agent-to-tools and agent-to-agent transport. They do not handle vendor-specific reasoning optimization.

No multi-vendor execution surface. The pain that builders feel today, when they want Claude Opus for strategic reasoning, Gemini Flash for high-volume worker tasks, Higgsfield for visual asset generation, and Seedance for video demos in the same project, is that they have to manually wire each vendor's SDK, manage each vendor's auth, normalize each vendor's output schema, and orchestrate the handoffs themselves. Multi-vendor adapter pain is the daily lived reality of every serious multi-agent builder.

2.3 The Core Insight

The tools for building software have never been more powerful. AI can write code, design systems, and deploy applications. But these capabilities are scattered, unstandardized, and difficult to access. Meanwhile, the number of AI agents in the world grows exponentially, while the ecosystem infrastructure that should support them does not exist.

CONFIDENTIAL & PROPRIETARY

NERIUM exists to be that infrastructure.

CLASSIFIED

3. The Solution: NERIUM

NERIUM bundles five "large products," each capable of standing alone as a company, into one integrated ecosystem. The cleanest analogy is structural: NERIUM aspires to be AWS plus Stripe plus DNS plus HTTP, but for the agent economy.

At the highest level, NERIUM creates a complete lifecycle for every agent:

> *Builder creates the agent. Marketplace lists and sells it. Banking processes payments to its creator and from its consumers. Registry maintains its identity and trust. Protocol routes its communication across model vendors.*

The pillars compound. Every agent built in Builder becomes potential supply for Marketplace. Every active Marketplace transaction generates Banking volume. Every Banking-active agent benefits from Registry verification. Every cross-vendor agent needs Protocol routing. The flywheel is not theoretical, it is structural.

The fundamental difference between NERIUM and any AI platform that exists today: NERIUM does not compete with AI companies. NERIUM does not build models. NERIUM is the infrastructure layer that every AI company and every agent needs. Whether one large model wins or one thousand small models share the market, all of them route their traffic through the same kind of plumbing. NERIUM is that plumbing.

Anthropic, Google, OpenAI, Meta, Mistral, and every vendor that comes after them sit above NERIUM in the application stack and beneath NERIUM in the routing stack. They produce the intelligence. NERIUM produces the substrate.

CLASSIFIED

CONFIDENTIAL / PROPRIETARY

4. Five Platform Pillars

PILLAR	NAME	ANALOGY	CORE FUNCTION
1	NERIUM Builder	Minecraft, with a JRPG wrapper	Construct agents and full applications from a single sentence through a gamified, multi-agent pipeline
2	NERIUM Marketplace	Marketplace platform of national scale (the Indonesian reader will recognize Tokopedia)	Open platform for buying and selling AI agents from any source
3	NERIUM Banking	Stripe	Payment infrastructure for live agent execution and settlement
4	NERIUM Registry	DNS	Identity, verification, and trust scoring for agents
5	NERIUM Protocol	HTTP	Cross-vendor communication standard that preserves each model's native conventions

Each pillar is an independent value proposition. Marketplace does not require Builder to function. Banking does not require Marketplace to process payments. Registry adds value to agents from any source. Protocol applies to every agent in the world, not only those born inside the NERIUM ecosystem.

When all five operate together, they create a network effect that is difficult to replicate: a complete environment where agents are built, traded, paid, trusted, and routed inside a single coherent infrastructure. The compound value of the five pillars is materially greater than the sum of any individual pillar, and the lock-in of running an agent through all five is structural rather than artificial.

CONFIDENTIAL / PROPRIETARY

4.5 Hackathon Submission Status (April 2026)

This section exists because honest framing wins judges and earns durable trust with future readers. Every claim in this PRD that describes NERIUM as currently shipping a capability is grounded in the artifacts described below. Every claim that describes a future capability is explicitly framed as future. The discipline is non-negotiable.

4.5.1 Build Context

NERIUM was constructed during the Cerebral Valley plus Anthropic "Built with Opus 4.7" hackathon in April 2026. Hard deadline is Monday April 27, 2026 at 07:00 WIB (April 26, 2026 at 8:00 PM EDT). The submission target is Monday April 27, 2026 at 06:00 WIB, with one hour of safety buffer. The submission format is a 3-minute demo video (maximum), a 100-to-200 word written summary, and a public GitHub OSS link under MIT license at `github.com/Finerium/nerium`.

The build was solo. One founder, one repository, one continuous orchestration arc from V0 through V6 handoff documents. No co-founder, no team, no employees. The throughput multiplier was Claude itself: 54-plus specialist Claude Code agents authored under formal Hephaestus prompt files in `.claude/agents/`, orchestrated through a Pythia modular contract discipline, validated by Nemea regression suites, logged daily by Ananke.

4.5.2 Recursive Automation Thesis

NERIUM built itself by running, one last time manually, the exact workflow that NERIUM Builder automates. Every step the founder performed by hand during the V1 through V6 orchestration is precisely what Builder collapses into conversation. The handoff documents in `_meta/orchestration_log/` are not just project artifacts, they are the source material for Builder's automated pipeline. The recursion is the pitch, and the pitch is true.

This frame appears throughout the submission surface: the README, the demo video narration, the Twitter announcement, the Discord post, and this PRD. One frame, no dilution. The product's origin story is the product's pitch, because the workflow that built NERIUM is literally what NERIUM Builder replaces.

CONFIDENTIAL / PROPRIETARY

4.5.3 What Ships at Submission

The following artifacts are present in the submission codebase at `github.com/Finerium/nerium`:

1. **54-plus specialist Claude Code agent prompt files** in `.claude/agents/`, each authored as a formal Hephaestus output, each carrying its own mandatory reading bundle and contract discipline.
2. **97 AI-generated game world assets** produced through Nano Banana Pro generation (Gemini 3 Pro Image Preview, accessed via Google AI Studio and gemini.google.com) followed by a rembg plus Canva background-removal pipeline. Asset inventory lives at `_Reference/ai_generated_assets/`. The breakdown is 78 PNG transparent assets (56 clean rembg outputs plus 22 Canva re-processes for assets where the rembg birefnet-general model produced quality regressions) and 19 JPG assets that did not require background removal.
3. **13 Phaser game world scenes** rendered at the `/play` route. Scene structure includes Apollo Village (the eight-pillar landmark hub), Caravan Road (the inter-pillar transit corridor), Cyberpunk Shanghai (the Builder Workshop world), and nine sub-area scenes that handle pillar-specific interactions.
4. **8 NERIUM-pillar landmarks** placed throughout Apollo Village as discoverable interactive zones. Each landmark routes the player to its corresponding pillar interface.
5. **Sekuri agent tier templates** at `public/sekuri/builder_templates/` covering small (4 agents, 5-minute build), medium (8 agents, 15-minute build), and large (14 agents, 45-minute build with multi-vendor orchestration).
6. **Multi-vendor selection UI** (the ModelSelectionModal component) presenting all eight supported vendors at the Builder entry point.
7. **Stripe Banking integration** wired in test mode, with the full payment flow shippable but executing against the test environment until Stripe Atlas activation completes post-submission.
8. **Public open source repository** under MIT license at `github.com/Finerium/nerium`. Reproducibility for judges and future readers is a first-class concern.

4.5.4 Theatrical Demo Plus BYOK Pattern

At submission scope, the Builder live-runtime invocation is not generally available. Instead, two paths exist:

CONFIDENTIAL / PROPRIETARY

Theatrical demo (default). The Builder demo at submission plays a pre-recorded theatrical sequence that mirrors the genuine multi-agent execution: the user submits a prompt, Sekuri classi-

fies the project complexity, the multi-vendor selection modal appears, the user picks vendors (or accepts Auto), and the agent cascade animates against an in-game scene with real spawn-time timing. The theatrical mode exists because the live runtime requires Anthropic API credits at scale and per-vendor billing accounts that have not yet activated. The theatrical sequence is not a fake. It is a recording of the actual workflow that ran during the submission build itself, surfaced as an interactive demo so any judge or reader can see what the live version will look like at full activation.

Bring-your-own-key (BYOK) live invocation. Judges and reviewers who want to validate the live runtime can paste their own Anthropic API key into the Builder interface. With a key present, the Builder switches from theatrical to live mode and executes a small Sekuri tier (typically the four-agent small template) end-to-end against the user's own API allocation. This pattern keeps the submission honest: the live runtime exists, it works, and any reviewer can validate it without trusting any claim the submission makes about itself.

4.5.5 Multi-Vendor Showcase, Anthropic-Only Invocation

The multi-vendor selection UI presents all eight supported vendors (Anthropic, Google Gemini, OpenAI, Higgsfield, Seedance, Meta, Mistral, and Auto). At submission, only the Anthropic invocation path is wired through to live execution. The other seven vendors appear in the UI as showcased options that surface their per-agent vendor overrides and their cost projections, but their runtime adapters ship dormant. This is a deliberate honest-claim choice: the UI is final, the routing logic is final, and the adapter framework is final, but the live execution against non-Anthropic vendors activates after per-vendor billing accounts open post-launch. The dormant adapters are not vaporware. They are wired, they have their schemas defined, and they pass their unit tests against mock vendor responses. Live activation is an account-onboarding milestone, not a software-engineering milestone.

4.5.6 Deployment State

At submission, NERIUM runs on Vercel free tier. This is sufficient for a hackathon demo and judges can access the deployment without latency concerns. The production deployment target is Hetzner dedicated infrastructure, which activates post-launch when the founder's Stripe Atlas onboarding completes and the per-vendor billing relationships go live. The Vercel deployment is canonical for the submission window. The Hetzner migration is part of the post-submission roadmap and does not block any submission claim.

CONFIDENTIAL / PROPRIETARY

4.5.7 Honest Disclaimers, Summarized

To make the constraint surface explicit:

1. Stripe is in test mode at submission. Live mode activates after Atlas onboarding completes.
2. Multi-vendor UI presents eight vendors. At submission, the live invocation path is wired exclusively for Anthropic; the other seven vendor adapters ship dormant.
3. Theatrical Builder demo is the default. BYOK option exists for live validation.
4. The 97 AI-generated assets are real and shipped. They were produced through Nano Banana Pro plus a rembg-and-Canva post-processing pipeline, not authored by hand.
5. The deployment is on Vercel free tier. Hetzner migration is post-launch.
6. The 54-plus specialist agents are real and the prompt files are committed. The orchestration arc that produced them is documented in `_meta/orchestration_log/`.

This list lives at the top of the README. It belongs in the PRD as well.

---

5. Pillar 1: Builder (Gamified Product Construction)

5.1 Overview

NERIUM Builder is the hero pillar. It wraps the entire process of building software into a game, and the game is played from inside a JRPG world wrapper that reframes "submitting a prompt" as "entering a workshop." The user's idea becomes a building they construct from the ground up. Each phase of software development maps to a phase of construction. Each agent in the pipeline appears as a character in the world. The orchestration is transparent, the controls are visible, and the experience is structured around momentum.

The framing is intentional. Coding tools that exist today operate as black boxes. The user types a prompt, the box returns code. There is no visibility into how decisions get made, what trade-offs get evaluated, or what an alternative path would have looked like. NERIUM Builder inverts this. Every agent that runs is shown. Every decision point that the system encounters is surfaced. The user retains the right to approve, redirect, or override at any point in the pipeline.

CONFIDENTIAL / PROPRIETARY

The thesis here is sharper than "developer experience." The thesis is that gamified prompting is the correct interface for non-technical users. People do not want to write prompts. People want to play. When the act of prompting is structured as a quest, with a visible map and a visible hero

and a visible objective, the prompting becomes addictive in the way that good games are addictive. The founder quote that opens this PRD makes the claim directly: NERIUM will make people addicted to creating prompts.

5.2 The JRPG World Wrapper

The Builder is not accessed through a form or a dashboard. The Builder is accessed through `/play`, the entry point to NERIUM's game world. From `/play`, the user enters Apollo Village, the central hub where eight NERIUM-pillar landmarks stand as discoverable interactive zones. The user walks (literally walks, in a Phaser-rendered 2D pixel-art world) to the landmark that corresponds to the pillar they want to engage. Builder lives inside the Builder Workshop, a Cyberpunk Shanghai aesthetic district reachable through Caravan Road from Apollo Village.

The thirteen Phaser scenes at `/play` are not decoration. They are the navigation primitive. Every interaction with NERIUM happens in-world. Marketplace is a literal market. Banking is a literal bank. Registry is a literal records office. Builder is a literal workshop. The metaphor compounds: the platform is not a website with pages, it is a city with districts, and the user walks between them.

This choice is deliberate and rests on a specific bet. The bet is that gamification, when done at JRPG-grade fidelity rather than dashboard-cosmetic fidelity, will attract a class of users who never otherwise engage with developer infrastructure. Indonesian high school students who play Pokémon Emerald will recognize the affordance immediately. Western users who grew up with Stardew Valley or RuneScape will recognize it. The interface is not novel for the sake of novelty. It is familiar to a generation that grew up inside game worlds, and it is the correct interface for that generation when the underlying activity (prompting AI to build software) has not yet found its native form.

5.3 The Building Metaphor

Inside the Builder Workshop, the actual product construction is framed through the metaphor of building a skyscraper. This metaphor was chosen because it is universally understood, naturally sequential, and maps cleanly to the actual stages of software development.

CLASSIFIED

CONFIDENTIAL / PROPRIETARY

CONSTRUCTION PHASE	PRODUCT PHASE	WHAT HAPPENS
Imagining the building	Ideation and brainstorming	The user describes their vision. AI characters help sharpen it.
Buying the land	Market validation	AI researches market, competitors, and feasibility.
Drawing the blueprint	Architecture and planning	AI generates system design, feature list, and agent structure.
Hiring the crew	Agent assembly	Specialist AI agents are assigned roles and responsibilities.
Laying the foundation	Core development	Agents write infrastructure, authentication, database, API.
Building the floors	Feature development	Each floor represents one feature module. Agents build, test, iterate.
Interior design	UI and UX refinement	Frontend, styling, user experience polish.
Safety inspection	QA and security	Test agents run checks. Security agents scan for vulnerabilities.
Grand opening	Deployment and launch	The product deploys and becomes accessible to real users.

The metaphor is more than visual flavor. It serves as a cognitive scaffold. Users who do not know what "deployment" means understand "grand opening" immediately. Users who do not understand "QA" understand "safety inspection." The metaphor lowers the cognitive cost of every step in the pipeline, and it does so without dumbing the underlying activity down. The crane is real. The crew is real. The floors are real. The mapping is consistent end-to-end.

5.4 The Sekuri Agent Tier System

The most concrete technical artifact of NERIUM Builder, and the one that replaces v1.0's vague three-tier description, is the Sekuri agent. Sekuri is the Builder's intake and routing specialist. Every prompt the user submits flows through Sekuri first. Sekuri performs three actions in sequence: classifies project complexity, selects the appropriate template, and presents the multi-vendor configuration to the user for approval.

CONFIDENTIAL / PROPRIETARY

Sekuri ships three pre-canned templates, each representing a different scale of project. The templates are not aspirational. They are concrete JSON files committed at `public/sekuri/builder_templates/`.

Small tier (4 agents, 5 minutes, USD 0.50 estimated cost).

The small tier handles single-feature, single-page projects with no authentication and no payment. Examples: a landing page with a signup form, a todo app with local storage, a single-page calculator with history, a weather widget, a simple validated contact form. The tier ships two parallel groups: P1 contains `athena_scaffold` and `thalia_ui`, P2 contains `proteus_logic` and `harmonia_polish`. All four agents run on Anthropic Opus 4.7. The terminal spawn count is 2. The cost projection is USD 0.50.

Medium tier (8 agents, 15 minutes, USD 2.00 estimated cost).

The medium tier handles multi-page projects with basic authentication, simple billing, and single deployment targets. Examples: a mid-tier SaaS dashboard with auth and billing, a blog platform with CMS and Stripe subscription, a freelancer portfolio with admin panel, a task management app with team collaboration, a newsletter platform with email integration. The tier ships three parallel groups: P1 contains scaffold, UI, and database; P2 contains logic, auth, and billing; P3 contains polish and QA. The agents split between Opus 4.7 (strategic agents: scaffold, database, QA) and Sonnet 4.6 (worker agents: UI, logic, auth, billing, polish). The terminal spawn count is 4. The cost projection is USD 2.00, all on Anthropic.

Large tier (14 agents, 45 minutes, USD 8.00 estimated cost across four vendors).

The large tier handles multi-tenant projects with complex billing, multi-vendor execution, and real-time requirements. Examples: a marketplace SaaS for indie agent creators, a marketplace platform of national scale (the multi-vendor e-commerce class), a real-time collaborative IDE with multi-tenant orgs, a production-grade banking app with Stripe Connect and AML compliance, an agent marketplace with trust score and escrow billing.

The large tier is the showcase for multi-vendor orchestration. Its `per_agent_vendor_overrides` configuration distributes work across four vendors:

CONFIDENTIAL / PROPRIETARY

AGENT	VENDOR	ROLE
`athena_scaffold`	Anthropic Opus 4.7	Strategic scaffolding
`demeter_db`	Anthropic Opus 4.7	Database design
`thalia_marketplace_ui`	Anthropic Sonnet 4.6	Marketplace UI
`proteus_protocol`	Anthropic Opus 4.7	Protocol logic
`tyche_identity`	Anthropic Sonnet 4.6	Identity and auth
`rhea_payment`	Google Gemini Pro	Payment flows
`phoebe_trust`	Google Gemini Flash	Trust scoring
`morpheus_adapter`	Anthropic Sonnet 4.6	Vendor adapter
`urania_blueprint_assets`	Higgsfield	Visual asset generation
`dionysus_demo_video`	Seedance	Demo video rendering
`hecate_orchestration`	Anthropic Opus 4.7	Pipeline orchestration
`harmonia_polish`	Anthropic Sonnet 4.6	Polish pass
`nemea_qa`	Anthropic Opus 4.7	QA validation
`kalypso_landing`	Anthropic Sonnet 4.6	Landing page

The large tier ships four parallel groups (P1 through P4) with explicit dependency blocking. P1 (scaffold, database, identity, orchestration) blocks no one. P2 (UI, protocol, payment, trust) waits on P1. P3 (visual assets, video, adapter) waits on P2. P4 (polish, QA, landing) waits on P3. The terminal spawn count is 6. The cost projection sums across vendors: USD 5.50 Anthropic, USD 1.20 Google, USD 0.80 Higgsfield, USD 0.50 Seedance.

This is what "Tier 1 Advisor / Tier 2 Leads / Tier 3 Workers" from PRD v1.0 collapsed into when the actual templates shipped. The hierarchy is real, but it is named by mythological role rather than tier number, and the routing is concrete rather than abstract.

CONFIDENTIAL / PROPRIETARY

5.5 The Multi-Vendor Selection Moment

After Sekuri classifies project complexity, the Builder surfaces the ModelSelectionModal. This modal is the user's primary point of agency over the pipeline. It presents all eight supported vendors as selectable options:

1. **Anthropic** (Opus 4.7, Sonnet 4.6, Haiku 4.5)
2. **Google Gemini** (Pro, Flash, Ultra)
3. **OpenAI** (Codex, GPT-5)
4. **Higgsfield** (visual generation)
5. **Seedance** (video generation)
6. **Meta** (Llama 3.1 405B and 70B)
7. **Mistral** (Large and Mixtral)
8. **Auto** (Sekuri picks per-agent based on tier configuration)

The user can accept the tier's default routing (Auto, which uses the per-agent vendor overrides from the matched template), or override per-agent for any agent in the roster. Cost projections update live as the user changes selections. The visual layout is a grid of vendor cards, each card showing the vendor's logo, available models, cost-per-token, and the agents currently routed to that vendor.

This moment is the protocol pillar made tangible. Section 9 covers Protocol in depth. From the user's perspective in the Builder, Protocol shows up as "Pick your vendors," and that's exactly the right framing.

5.6 The Agent Cascade

After the user approves the multi-vendor configuration, the agent cascade begins. This is the visual centerpiece of the Builder demo. The selected agents appear as characters in the workshop scene, each one animated to spawn at its dependency-resolved position. The user watches the cascade unfold in-world, with real spawn-time timing and dependency-blocked groups visibly waiting for their parents to complete.

For a small tier project, the cascade lasts approximately five minutes and produces a deployable app. For a medium tier, it lasts fifteen minutes. For a large tier with multi-vendor routing, it lasts approximately forty-five minutes. The per-tier durations are estimates committed in the JSON templates and are tuned against real-world Sekuri runs.

CONFIDENTIAL / PROPRIETARY

At submission, the cascade renders as the theatrical demo described in Section 4.5.4. The animation is real, the timing is real, the agent roster is real. The execution underneath the

animation is either a real Anthropic-only run (when the user supplies their own API key) or a recorded run from the build itself (the default theatrical mode).

5.7 The Six-Phase Automation Pipeline

Underneath Sekuri, the actual automation pipeline runs through six phases. The phases are inherited from the workflow that the founder used to ship NERIUM itself, and they are exactly what Builder collapses into a single conversational interface.

Phase 0: Genesis. A brainstorming agent guides the user through structured idea exploration. Output: an idea document covering product vision, target user profile, core feature description, data sources, technical preferences, and success criteria. This phase exists in plain conversation rather than form-fill.

Phase 1: Architecture. An architecture agent produces a complete technical framework: system architecture, data schemas, API specifications, ML and AI pipeline definitions, feature specifications, design system, coding standards, file structure, development phases, testing strategy, and deployment plan. This document is the contract that all subsequent agents work against.

Phase 2: Implementation Planning. A planning agent decomposes the framework into per-module implementation prompts. Each prompt includes module-level coding contracts that define interfaces, data types, and inter-module dependencies. Each prompt is designed as a self-contained instruction so that downstream coding agents can execute without context from any other agent.

Phase 3: Development. Coding agents pick up implementation prompts and build modules in parallel, respecting the dependency-blocking groups defined by the tier template. Infrastructure first, then backend, then frontend. Every agent emits a handoff report covering progress made, decisions taken, and potential blockers, and the report flows into the next agent in the chain.

Phase 4: Integration and QA. Test agents run integration tests across modules, end-to-end tests, and security audits. Bug reports flow back into a fix-iteration loop until QA agents sign off.

Phase 5: Deployment and Documentation. Deployment agents configure cloud infrastructure and run continuous-integration pipelines. Documentation agents produce README files, API documentation, architecture guides, developer guides, and user manuals.

Phase 6: Validation and Handoff. A final validation agent verifies the entire product against the framework specification and produces a final project report. The user receives a deployable artifact.

CONFIDENTIAL / PROPRIETARY

The critical property of this pipeline: at every transition point between phases, the user can review, modify, or approve outputs before the pipeline advances. The system is fully automated,

but it remains human-in-the-loop at every strategic decision. The user is not orchestrating handoffs by hand. They are approving or revising, and the platform handles the rest.

5.8 Honest-Claim Disclaimers for Builder

At hackathon submission scope, Builder's live runtime is whitelisted to a small set of templates and vendor combinations. The full multi-vendor live execution against Higgsfield, Seedance, Meta, Mistral, and OpenAI activates post-launch when per-vendor billing accounts open. The theatrical demo described in Section 4.5.4 is the default presentation surface. The BYOK option lets reviewers validate the live Anthropic path. Production-grade output for genuine national-scale e-commerce platforms is aspirational future capability and is explicitly framed as such everywhere it appears in this document.

6. Pillar 2: Marketplace (Open Agent Economy)

6.1 Overview

NERIUM Marketplace is an open platform for buying and selling AI agents. It is not a complementary feature of Builder. It is a standalone product with an independent value proposition, and it operates regardless of where the agents being traded were originally built.

The structural analogy: Builder is a fully equipped professional kitchen where one can cook anything from raw ingredients. Marketplace is not a cookbook for that kitchen. Marketplace is a separate supermarket. People go to the supermarket to buy and sell ingredients, finished dishes, and recipes, regardless of which kitchen produced them.

In-game, the Marketplace is a literal market district reachable from Apollo Village along the Caravan Road. Vendors operate stalls. Agents are stocked as inventory. The pixel-art aesthetic continues seamlessly from the Builder Workshop, and the metaphor of a market remains stable: people walk in, browse, transact, and leave.

6.2 Open Platform Philosophy

CONFIDENTIAL / PROPRIETARY

Marketplace accepts agents from any source. Agents built with NERIUM Builder, agents hand-written by developers, agents produced with Cursor, Claude Code, Replit, Bolt, Lovable, or any

CLASSIFIED

other tool, all of them are listable provided they meet a minimum quality bar. This is a deliberate design choice and it rests on two arguments.

First, restricting supply to Builder-produced agents would severely cap Marketplace adoption. Opening the platform admits supply from the entire ecosystem: hardcore developers who write agents from scratch, vibe coders who ship fast and want to monetize immediately, AI companies that already ship solutions and want a new distribution channel, and Builder users who happen to publish there.

Second, the open platform stance reinforces the case for Banking, Registry, and Protocol, because all three become relevant to every agent in the world rather than only to agents born inside the NERIUM ecosystem. The pillars compound stronger when each one operates universally.

6.3 Living Templates

The differentiating feature of Marketplace is the concept of living templates. Unlike conventional marketplaces that sell static products, templates on NERIUM Marketplace can be routed directly into the Builder pipeline for customization.

The flow: a creator publishes an agent template, for example "Smart Agriculture Agent for Chili Farming," with full source code and configuration. A buyer purchases the template and then instructs the platform: "Adapt this for grape farming." NERIUM Builder runs its automated pipeline: it reasons about the domain shift between chili and grape (growth cycles, soil conditions, disease vectors, harvest metrics), adjusts the prediction models, modifies the sensor parameters, and updates the recommendation logic. The buyer has not started from zero. The buyer has started from a proven foundation and ridden the customization pipeline to their specific use case.

This makes Marketplace simultaneously two kinds of platform: a place to buy ready-to-use agents, and a place to buy customizable foundations. It also solves the cold-start problem, because creators can use Builder to author templates, then publish them directly. Supply and demand form inside one ecosystem.

6.4 The Restaurant Automation Example

A concrete grounding for the Marketplace value proposition: consider a small restaurant owner who wants to automate inventory tracking, supplier ordering, and shift scheduling. Today, the path is to either hire a developer (six-digit cost, multi-month timeline) or adopt a generic SaaS product that does not fit the restaurant's specific workflow. Neither option works for most restaurant owners.

CONFIDENTIAL / PROPRIETARY

With NERIUM Marketplace, the owner browses for "restaurant operations agent," finds a template tuned to mid-sized full-service restaurants, purchases it, and then runs the Builder customization pipeline against their specific menu, supplier list, and staffing structure. The output is an operational agent, not a generic SaaS product, and the marginal cost of customization is small compared to building from scratch.

This example is mentioned in v1.0 in passing. It deserves more weight, because it represents the broader thesis: the long tail of small businesses that cannot afford bespoke software development becomes addressable when the marginal cost of customization collapses. Every restaurant, every clinic, every boutique, every small farm, every independent retailer becomes a potential Marketplace customer.

6.5 Rating and Review System

Every agent on Marketplace carries a comprehensive rating system: user reviews, automated performance scores, usage history, and reliability track record. The system integrates with NERIUM Registry (Section 8) so that ratings reflect actual quality rather than marketplace manipulation.

6.6 Competitive Moat

The Marketplace's competitive moat is reinforced by the customization engine being tied to the platform. A buyer cannot extract the same modification quality by purchasing a template and editing it themselves with a generic AI tool. They need the Builder pipeline to obtain a seamless adaptation. This binds Marketplace value to the broader NERIUM platform.

Revenue layering is also natural: a transaction fee from the initial purchase, plus an AI usage fee from the customization run. One sale generates revenue across two pillars.

7. Pillar 3: Banking (Financial Infrastructure)

7.1 Overview

CONFIDENTIAL / PROPRIETARY

NERIUM Banking is financial infrastructure built specifically for the agent economy. Once agents are deployed and running in the real world (monitoring IoT sensors, handling 24/7 customer

service tickets, analyzing market data in real time, managing inventory), they consume resources continuously. APIs, compute, third-party services. Every consumption event needs a payment system that handles usage-based billing for living agents.

In-game, the Banking pillar is a literal bank in Apollo Village, reachable along Caravan Road. The interior of the bank shows live transaction flows as animated coin sprites moving between agent wallets, creator accounts, and the platform treasury. The visualization is not decoration. It is the actual UI for transaction monitoring, rendered in the same pixel-art aesthetic as the rest of the world.

7.2 The Future Context

Looking forward, the number of AI agents operating live through APIs will be very large. Every industry will have agents running continuously: agriculture, healthcare, finance, logistics, customer service, and hundreds of other domains. Every one of these agents consumes and produces value continuously, and every interaction requires a financial transaction.

Today, payment infrastructure for agents uses generic solutions that were not designed for this use case. Stripe handles SaaS subscriptions excellently. Stripe Connect handles marketplace payouts. Neither product was shaped specifically for things like micropayments per API call from an agent, revenue splits between creators and platforms, escrow for agent-to-agent transactions, credit systems for agents that have not yet generated revenue, and settlement for agents operating across currencies and jurisdictions.

NERIUM Banking is the answer to all of those gaps, in one platform.

7.3 Core Functions

Usage-Based Billing. Creators who sell agents via API set a price per usage event. Buyers who consume the agent pay based on actual consumption. NERIUM Banking handles all metering, invoicing, and settlement.

Agent Wallets. Every agent operating in the ecosystem has an "account" inside NERIUM Banking. Revenue generated by the agent accumulates in this wallet. Creators can withdraw to traditional bank accounts or use the balance to purchase other services in the ecosystem (a credit-pool effect that retains capital inside NERIUM).

Transaction Processing. The platform processes transactions between buyers and sellers, between agents and third-party services, and between agents that consume each other's services. The agent-to-agent settlement case is the future-shaped one: when one agent calls another agent's API, the payment routing happens through Banking with no human in the loop.

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

Cost Transparency. Users get full visibility into projected costs and actual costs, with the ability to set budgets and receive alerts as they approach limits. Cost transparency is non-negotiable, because trust dissolves the moment a user receives an unexpected bill.

7.4 Honest-Claim Disclaimer for Banking

At submission, NERIUM Banking is wired against Stripe in test mode. The full payment flow (metering, invoicing, settlement, withdrawal) is functional end-to-end against the test environment. Live mode activates after Stripe Atlas onboarding completes, which is in progress as of submission and is expected to land within weeks of the hackathon close. The Banking pillar's live execution is therefore in a known waiting state, not a software-engineering blocker. Reviewers can validate the test-mode flow in the submission demo and trust that the live-mode flow will operate identically once Atlas activates.

7.5 Strategic Role

NERIUM Banking locks users in at the financial level. Once an agent's money flow runs through NERIUM infrastructure, moving to another platform requires migrating the entire financial history, payment relationships, and settlement mechanisms. This creates a significant switching cost and reinforces NERIUM's position as essential infrastructure.

The strategic point is not "lock-in" as a hostile mechanic. The strategic point is "lock-in" as a structural property of running financial rails. Stripe is hard to leave because Stripe runs the rails. NERIUM Banking is hard to leave because it runs the rails for agents.

---

8. Pillar 4: Registry (Identity and Trust Layer)

8.1 Overview

NERIUM Registry gives every agent operating in the world an identity. A persistent record. Verified capabilities. A trust score. A track record. Information about its creator. Without this, the agent economy is unregulated open territory, and unregulated open territory is incompatible with enterprise adoption.

CONFIDENTIAL / PROPRIETARY

In-game, the Registry is a records office in Apollo Village. The interior shows ledgers, identification documents, and verification badges. The aesthetic continues from the rest of the world.

8.2 The Problem

Anyone can create and publish an AI agent. There is no standardized mechanism today to answer the fundamental questions: Does this agent really do what it claims? Is this agent safe? Who built it? What is its track record? Has it been audited?

When the agent economy reaches a scale of millions of agents operating concurrently, these questions become critical. Organizations will not deploy agents for important tasks without a reliable verification mechanism, and individual users will not trust agents from unknown publishers. The current state of the agent market, where trust is established by direct messaging the creator on Twitter, is not viable at scale.

8.3 Core Functions

Agent Identity. Every registered agent has a unique identity covering name, version, capability description, creator, creation date, and update history. The identity is persistent and verifiable.

Trust Score. A trust algorithm computes a score across several dimensions: usage history, user ratings, security audit results, uptime, output accuracy, creator reputation. The trust score updates dynamically based on actual performance, not on initial claims.

Verified Capabilities. Agents can submit capability claims that are then verified through automated testing or third-party audit. Verification status displays publicly so users can distinguish between claimed and verified capabilities. This addresses the fundamental "does it actually work" question that today requires direct testing.

Audit Trail. Every important agent activity is logged: when it was created, when it was updated, who has used it, what incidents have occurred, and how those incidents were handled. The audit trail is the historical substrate that enables trust evaluation.

8.4 The DNS Analogy

As DNS maps domain names to IP addresses and operates as fundamental internet infrastructure (without which the web cannot function), NERIUM Registry maps agent identities to capabilities and trust levels. When someone wants to use any agent from any source, they check Registry first: Is this agent legitimate? What is its rating? Has it been audited?

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

The analogy compounds with the rest of the platform. Just as DNS is invisible to the average web user but essential to every web request, Registry is intended to be invisible to the average NERIUM user but essential to every transaction.

9. Pillar 5: Protocol (Interoperability Standard)

9.1 Overview

NERIUM Protocol is the communication layer that lets AI agents from different models and vendors collaborate optimally without losing the unique characteristics of each model. It is the substrate that makes the multi-vendor selection moment described in Section 5.5 possible at a technical level. Without Protocol, "Pick your vendors" is a marketing claim. With Protocol, it is a working pipeline.

9.2 The Fundamental Problem

Each AI vendor has a distinct prompting style and reasoning convention. Anthropic's Claude is optimal when receiving instructions in XML-tag format, with explicit reasoning scaffolds and CLAUDE.md context files. Google's Gemini follows a different convention, more verbose in some areas and more terse in others. OpenAI's models follow yet another convention. As more AI vendors enter the market, large and small, the fragmentation deepens.

When users want to build systems that span multiple AI models (Claude Opus for backend reasoning, Gemini Flash for high-volume worker tasks, OpenAI Codex for code generation, Higgsfield for visual assets, Seedance for video), they currently have to manually understand each vendor's optimal communication patterns and orchestrate the cross-vendor handoffs themselves. This is the multi-vendor adapter pain that every serious multi-agent builder lives with.

9.3 Approach: Translation Layer, Not Single Language

NERIUM Protocol is not an attempt to force all AI vendors to speak the same language. That approach would erase the unique strengths of each model. Instead, Protocol acts as a translation and adapter layer.

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

The cleanest analogy is diplomatic interpretation. A Japanese speaker continues to speak Japanese. An Arabic speaker continues to speak Arabic. The interpreter between them knows how to convey the meaning from one language to the other in a way that preserves intent. Claude continues to receive XML tags. Gemini continues to receive its preferred format. Protocol does not change the native language of any model. Protocol handles how messages translate between them so the result is optimal at every endpoint.

9.4 The Eight-Vendor Surface

NERIUM Protocol explicitly supports eight vendor surfaces at submission, with the per-vendor adapters wired and tested:

VENDOR	AVAILABLE MODELS	PRIMARY USE CASES
Anthropic	Opus 4.7, Sonnet 4.6, Haiku 4.5	Strategic reasoning, code synthesis, long-context orchestration
Google Gemini	Pro, Flash, Ultra	High-volume worker tasks, multimodal reasoning, fast iteration
OpenAI	Codex, GPT-5	Code generation, broad-coverage assistance
Higgsfield	Visual generation models	Image asset generation, sprite production, blueprint visuals
Seedance	Video generation models	Demo video rendering, motion content, scene generation
Meta	Llama 3.1 405B and 70B	Self-hosted reasoning, open-weights workloads
Mistral	Large, Mixtral	Cost-efficient reasoning, specialized worker roles
Auto	Sekuri picks per-agent based on tier	Default routing for users who do not want to configure manually

The list is not exhaustive of vendors that exist in the market. It is exhaustive of vendors that NERIUM Protocol ships adapters for at submission. New vendors can be added through the adapter framework without changes to the core Protocol logic.

CONFIDENTIAL / PROPRIETARY

9.5 The Sekuri Large Template as Concrete Example

The Sekuri large tier described in Section 5.4 is the concrete demonstration of multi-vendor orchestration through Protocol. Its `per_agent_vendor_overrides` configuration distributes work as follows:

Anthropic for strategic agents. Athena (scaffold), Demeter (database), Proteus (protocol logic), Hecate (orchestration), and Nemea (QA) all run on Opus 4.7. Thalia (marketplace UI), Tyche (identity), Morpheus (adapter), Harmonia (polish), and Kalypso (landing) run on Sonnet 4.6. The ten Anthropic-routed agents handle reasoning, code synthesis, and orchestration where Claude's strengths are highest.

Google Gemini for worker agents. Rhea (payment) runs on Gemini Pro. Phoebe (trust) runs on Gemini Flash. The two Gemini-routed agents handle high-volume work where Gemini's cost-per-token and throughput are optimal.

Higgsfield for visual assets. Urania (blueprint assets) runs on Higgsfield to generate the in-game and in-product visual assets the project needs. This is a pure specialization choice: visual generation is what Higgsfield does best.

Seedance for video demos. Dionysus (demo video) runs on Seedance to generate the demonstration videos the project ships with. Again, pure specialization.

The cost projection across the four vendors sums to USD 8.00 for a single 45-minute large-tier run. Compare this to the cost of routing the same work entirely through Anthropic Opus 4.7, which would be substantially higher and would not access the visual or video generation surfaces at all. Protocol enables this kind of distribution by handling the cross-vendor handoffs invisibly.

9.6 Industry Context

The current landscape of agent communication protocols is consolidating around several standards. The Model Context Protocol (MCP), created by Anthropic and now governed by the Linux Foundation, standardizes how agents interact with external tools and resources. The Agent-to-Agent Protocol (A2A), initiated by Google with more than 50 technology partners, standardizes peer-to-peer communication between agents. The Agent Communication Protocol (ACP) from IBM handles structured messaging in local environments.

NERIUM Protocol operates at a different and complementary layer. MCP handles agent-to-tools connections. A2A handles agent-to-agent communication. NERIUM Protocol handles communication optimization based on the specific model. Protocol ensures that when a Claude-based agent needs to delegate a task to a Gemini-based agent, the message is not just transmitted

CONFIDENTIAL / PROPRIETARY

(which A2A already handles) but translated into the format that is most optimal for the receiving model. Protocol sits above the transport standards, not in competition with them.

9.7 Honest-Claim Disclaimer for Protocol

At submission, the multi-vendor UI presents all eight vendors and the per-vendor adapter framework is wired and unit-tested. Live invocation against non-Anthropic vendors activates after per-vendor billing accounts open post-launch. The dormant adapters are not vaporware. They have schemas defined, mock-vendor unit tests passing, and routing logic in place. Live activation is an account-onboarding milestone, not a software-engineering one. The submission's honest framing on this point is non-negotiable: today, the multi-vendor showcase is real UI plus dormant runtime; tomorrow, the runtime activates one vendor at a time as billing relationships go live.

9.8 Long-Term Vision

If NERIUM Protocol is adopted broadly, it becomes the rail on which all agents in the world run. Every time an agent from one model needs to communicate with an agent from another model, they use NERIUM Protocol. This creates an infrastructure position that is difficult to displace, because the value of the network grows exponentially with each new agent that joins. Standards win when they become the default. NERIUM Protocol's path to default is through Builder, where the multi-vendor selection moment is the natural funnel that introduces every Builder user to Protocol whether they think of it that way or not.

10. User Journeys

10.1 Journey A: Builder (Building from Zero)

Consider Rina, a small business owner who runs a chain of three salons in Bandung. She wants to build a booking platform that lets her customers reserve appointments at any of her locations, see available time slots, choose preferred stylists, and pay online.

Rina opens NERIUM and walks her character through Apollo Village to the Builder Workshop in Cyberpunk Shanghai. She enters the Workshop and is greeted by the Sekuri agent. She describes her idea in plain language: "I want an app where my customers can book appointments at any salon location, see available slots, pick their favorite stylist, and pay online."

CLASSIFIED

Sekuri asks follow-up questions: How many locations does she have? What does the current booking process look like? What frustrations do her customers experience? Within fifteen minutes, they have a sharpened concept.

Sekuri classifies the project as medium tier (multi-page, basic auth, simple billing, single deployment), pulls the medium template, and presents the multi-vendor configuration. Rina accepts the default Anthropic-only routing (Opus 4.7 for strategic work, Sonnet 4.6 for workers). The cost projection shows USD 2.00 and a 15-minute runtime.

The Builder generates an architecture: a system with modules for customer accounts, location management, appointment scheduling, stylist profiles, payment processing, and notifications. Eight specialist agents are assembled, each with clear roles. Rina reviews everything, asks a few questions, adjusts a few feature priorities, and approves.

Construction begins. Over the next fifteen minutes, Rina watches her building rise. The foundation goes in (database and authentication). The first floor (appointment scheduling). The second floor (payment integration). The third (stylist profiles and notifications). At one point, the system asks her to choose between two notification approaches. She picks the simpler one. At another point, the QA agent flags a calendar logic edge case. She approves the fix.

At the fifteen-minute mark, the building is complete. Rina clicks "Grand Opening" and receives a link to her live booking platform. She shares it with her customers that same afternoon.

10.2 Journey B: Marketplace (Buying and Customizing)

Consider Budi, a farmer in Probolinggo who has heard about AI agents for precision agriculture. He is not a developer. He has never written code. But he has friends who have used NERIUM, and they showed him the Marketplace.

Budi enters Apollo Village and walks to the Marketplace district. He browses the "Agriculture and Farming" category. He finds an agent template called "Smart Agriculture Agent for Chili Farming" with a 4.7 rating across 230 reviews. The agent is designed to monitor IoT sensors in fields, predict optimal harvest timing, and recommend fertilization schedules.

Budi purchases the template. But he grows grapes, not chili. He instructs the platform: "Adapt this for grape farming."

NERIUM Builder takes over. It runs the customization pipeline: it reasons about the differences between chili and grape (growth cycles, soil conditions, pest profiles, harvest metrics), adjusts the prediction models, modifies the sensor parameters, and updates the recommendation logic.

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

Within a few hours, Budi has a customized grape-farming agent built on top of the proven chili-farming foundation, without writing a single line of code. He installs it on his sensors and lets it run.

10.3 Journey C: Banking and Registry (Live Agent Operations)

Consider Arif, a developer who has built a powerful stock-analysis agent specifically tuned to the Indonesian Stock Exchange (IDX).

Arif publishes his agent on NERIUM Marketplace and offers it as a paid API. NERIUM Registry assigns the agent a verified identity, complete with a trust score grounded in security audit results and prediction accuracy from the testing phase.

Users start consuming Arif's agent through the API. NERIUM Banking handles the entire financial cycle: per-API-call usage metering, monthly invoicing of consumers, settlement to Arif's wallet, and detailed revenue reporting. Arif does not have to build billing infrastructure himself. He focuses on improving the agent's prediction quality, while NERIUM handles the rest.

Over time, the agent's trust score rises based on accumulated positive ratings and reliability track record. The higher trust score attracts more users, which generates more revenue, which justifies further investment in agent quality. The flywheel runs.

10.4 Journey D: The Creator Path (Recursive)

Consider a fourth scenario, more specific to the platform's recursive thesis. A creator named Lia is an indie .skills author who currently sells her work through Twitter direct messages. She is tired of payment friction, of buyers ghosting after asking detailed pre-sale questions, of having no record of who bought what. She finds NERIUM and migrates her catalog.

She publishes seven of her .skills as agent templates on Marketplace. Registry assigns each one a verified identity and an initial trust score based on her existing creator reputation. Banking handles the payment layer. Within two weeks, her revenue is up 4x because the friction collapsed. She does not have to be a salesperson anymore. She is a creator, and the infrastructure does the rest.

This journey is the marketplace pillar at its most direct: a creator who already has product, who already has audience, who only lacked the rails. NERIUM gives her the rails.

CONFIDENTIAL / PROPRIETARY

11. Business Model and Monetization

NERIUM's monetization is designed to be transparent, scalable, and aligned with user success. The platform generates revenue through several mutually reinforcing channels.

11.1 AI Model Usage Fees (Builder)

Agents in NERIUM Builder are powered by external AI models from various providers (Anthropic, Google, OpenAI, Higgsfield, Seedance, Meta, Mistral). Users pay for the AI compute they consume. The platform acts as a managed layer above these APIs, handling all the complexity of model selection, prompt engineering, and orchestration. The platform applies a reasonable markup over API costs to cover infrastructure and platform services.

Users can choose which AI tier to use. More capable models cost more but produce better output. This creates a natural upsell path without forcing users into subscription tiers they do not need.

Bring-your-own-key (BYOK) runtime mode. Users can also supply their own API keys for any of the supported vendors. In BYOK mode, the markup on AI compute is removed and the platform monetizes through subscription instead. This is a deliberate flexibility: power users who manage their own vendor relationships are not penalized, and casual users who want managed billing get the convenience.

Theatrical demo mode (default at submission). For the hackathon submission window, the default Builder demo mode is theatrical (recorded multi-agent execution surfaced as an interactive demo) rather than live. The theatrical mode is free. The BYOK live mode requires the user to supply their own Anthropic API key. Post-launch, with Stripe Atlas activated and per-vendor billing accounts open, the platform-managed live runtime mode becomes the default and the theatrical mode is retained as a public preview surface.

11.2 Platform Subscription

A freemium model gives access to basic features: one project building, limited agent count, basic AI models. Paid tiers unlock more projects, higher agent limits, premium AI models, faster execution, priority support, and collaboration features.

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

TIER	PROJECTS	AGENT LIMIT	AI MODELS	INDICATIVE PRICING
Free	1	4 (small tier only)	Anthropic Opus 4.7 only	Free
Starter	5	8 (medium tier)	Anthropic full lineup	TBD per month
Pro	Unlimited	14 (large tier)	All eight vendors	TBD per month
Enterprise	Unlimited	Unlimited	All eight vendors plus custom routing	Custom

The pricing values are deliberately marked TBD as of submission. Final pricing locks in post-launch alongside Stripe Atlas activation. The tier structure itself is locked in the Sekuri tier system described in Section 5.4.

11.3 Marketplace Transaction Fees

Every Marketplace transaction generates a fee for the platform. This covers template sales, completed agent sales, and customization transactions. The fee structure follows the standard two-sided marketplace model: a percentage of each transaction split between the platform and operational costs.

11.4 Banking Processing Fees

Every transaction processed through NERIUM Banking generates a processing fee. This covers API usage transactions, settlement to creators, and inter-wallet transfers. The model mirrors how payment processors like Stripe generate revenue: a small percentage of each transaction that flows through the infrastructure.

At submission, Banking runs in Stripe test mode. Live processing fees activate after Stripe Atlas onboarding completes.

11.5 Registry Premium Services

Basic Registry services are free for all agents. Premium services include accelerated capability verification, third-party security audits facilitated by the platform, "Verified" badges that increase Marketplace visibility, and advanced analytics on agent performance.

CLASSIFIED
CONFIDENTIAL / PROPRIETARY

11.6 Protocol Licensing (Long-Term)

If NERIUM Protocol is broadly adopted, the long-term business model can include enterprise licensing for companies that want to integrate Protocol into their infrastructure, plus premium support and SLAs for large-scale Protocol use. This is explicitly future revenue, not submission-window revenue.

11.7 Flywheel Revenue

What makes NERIUM's business model strong is the flywheel effect: each pillar generates independent revenue, but activity in one pillar drives revenue in another. An agent built in Builder (generating usage fees) can be sold on Marketplace (generating transaction fees), then operate live (generating banking fees), require verified identity (generating registry fees), and communicate cross-vendor (using Protocol). One agent can generate revenue across all five layers.

The unit economics of each pillar in isolation must be defensible on their own. The compounding effect across all five is the multiplier.

12. Market Size and Opportunity

12.1 The Global AI Agent Market

The global AI agent market is projected to experience explosive growth over the next five years. Based on research from several leading analyst firms, market size estimates range from USD 42 billion to USD 52 billion by 2030, with CAGR between 41 and 46 percent from 2025.

According to MarketsandMarkets, the AI agent market is projected to grow from USD 7.84 billion in 2025 to USD 52.62 billion in 2030, with a CAGR of 46.3 percent. Capgemini reports that the agentic AI market was valued at USD 5.1 billion in 2024 and is expected to exceed USD 47 billion by 2030. IDC projects spending on agentic AI will exceed USD 1.3 trillion by 2029.

Several segments relevant to NERIUM show particularly high growth. The coding and software development segment is projected to grow at 52.4 percent CAGR. Multi-agent systems are projected to grow at 48.5 percent CAGR. Vertical AI agents are projected to grow at 62.7 percent CAGR.

CONFIDENTIAL / PROPRIETARY

12.2 Adjacent Markets

Beyond the direct AI agent market, NERIUM also operates at the intersection of several other large markets. The low-code and no-code development platform market. The payment processing market (led by Stripe and similar players). The cloud infrastructure market (led by AWS, GCP, Azure). The marketplace platform market (led by Shopify, Gumroad, and similar players). NERIUM's positioning at the intersection of all these markets creates a total addressable market substantially larger than the AI agent market alone.

12.3 Timing

The year 2026 marks a critical inflection point for AI agent adoption at the enterprise level. Gartner estimates that by 2026, nearly every business application will have an AI assistant, with 40 percent integrating task-specific agents within the following year, up from less than 5 percent in 2025. Protocol standards like MCP and A2A have reached sufficient maturity to indicate that the foundational infrastructure layer is starting to crystallize. NERIUM arrives at the right moment: early enough to define the category, late enough that market pull is clearly felt.

13. Target Audience

NERIUM serves several distinct audience segments, each interacting with different pillars of the platform.

13.1 Builder Audience

Aspiring founders and solopreneurs. People with product ideas who want to build a minimum viable product without hiring a development team. They may have business experience but little or no coding knowledge.

Small business owners. People who need custom software for their business but cannot justify the cost of hiring a development agency. The restaurant automation example in Section 6.4 lives here.

Product managers and designers. Professionals who understand what needs to be built but currently depend on engineering teams to execute. NERIUM Builder gives them a way to prototype and validate ideas independently.

CLASSIFIED
CONFIDENTIAL / PROPRIETARY

Students and learners. People who want to understand how software is built by watching the process happen transparently and instructively. The Indonesian student perspective is particularly relevant here. The founder of NERIUM is a first-year student at Politeknik Negeri Bandung. Students who grew up playing games are the natural first audience for a platform that uses game mechanics to teach software construction.

Technical founders. Even experienced developers can benefit from agent orchestration to accelerate development and handle tasks outside their core expertise. Builder is not exclusively for non-technical users. Builder is faster than hand-written code for many classes of project, regardless of the user's technical background.

13.2 Marketplace Audience

Agent creators and sellers. Developers, AI companies, and vibe coders who want to monetize agents they have built. The .skills authors selling on Twitter. The MCP server authors with unstarred GitHub repositories. The AI consultants who currently sell custom agents to enterprise clients.

Agent buyers. Businesses of all sizes seeking ready-to-use AI solutions or customizable templates for specific needs. Restaurant owners. Farmers. Clinics. Boutiques. The long tail of small businesses that the conventional software industry does not serve.

System integrators. Companies that build custom solutions for clients and need access to proven agent components.

13.3 Banking, Registry, and Protocol Audience

Enterprise. Large organizations operating dozens or hundreds of agents concurrently who need managed infrastructure for payment, identity, and interoperability.

AI companies. AI companies of all sizes who want their agents to operate, sell, and communicate within a standardized ecosystem.

Developer ecosystem. The developer community building on top of the agent ecosystem who need consistent tooling and standards.

CONFIDENTIAL / PROPRIETARY

14. Competitive Landscape and Differentiation

14.1 Direct Competitors (Builder)

Several platforms exist that use AI to help build software. NERIUM Builder differs fundamentally from all of them.

Lovable generates clean, attractive React code from natural-language prompts. But Lovable produces code, not running applications. Users still need to configure Supabase, manage row-level security policies, handle deployment, and understand technical concepts. Lovable is a tool for developers who want to move faster, not a platform for non-technical users who want to build products.

Bolt.new offers very fast prototyping in the browser with real-time preview. But it operates as a browser-based IDE where an AI pair-programmer assembles the stack, not as a gamified experience that guides users through the entire process. Users still need to understand development concepts.

Replit Agent has evolved into a full-stack platform with database, authentication, hosting, and 30-plus built-in integrations. The agent can work autonomously and test its own code. But Replit still feels like a developer environment, not an experience designed for non-technical users.

Cursor and Windsurf are IDE assistants that accelerate coding for developers. They are productivity tools, not end-to-end product-building platforms.

14.2 NERIUM Builder Differentiation

NERIUM Builder differs from all of the above in three fundamental ways. Each differentiator is grounded in shipped artifacts in the submission codebase.

Gamification as the interface. No other platform uses a JRPG world wrapper as the primary interface for software development. This is not a gimmick. It is a deliberate design choice that makes the process accessible, motivating, and intelligible to non-technical users. The shipped artifact is the thirteen Phaser scenes at `/play`, the eight pillar landmarks in Apollo Village, and the Caravan Road plus Cyberpunk Shanghai world structure. The metaphor of building a skyscraper provides an intuitive mental model for a process that is otherwise highly technical.

Transparent agent orchestration. Existing platforms operate as black boxes. Users type a prompt and get code back. They have no visibility into how decisions are made or what is happening behind the scenes. NERIUM Builder makes the entire agent system visible and controllable. Users see every agent, every task, and every decision, and they approve the important

CONFIDENTIAL / PROPRIETARY

ones. The shipped artifact is the Sekuri agent intake (with its classify-and-route logic exposed as user-visible JSON), the multi-vendor selection modal, and the real-time agent cascade visualization. Transparency is not aspirational here. It is shipped.

End-to-end scope with automated pipeline. Most AI coding tools handle one slice of the development process. NERIUM Builder covers the entire journey: from idea to validation to architecture to development to testing to deployment. Beyond that, the entire multi-agent pipeline that normally requires 100-plus manual steps is fully automated. The shipped artifact is the Sekuri tier system (small, medium, large) with concrete agent rosters, parallel execution groups, and dependency-blocked phases. The 54-plus specialist agents in `.claude/agents/` are the source material for this pipeline, and they were used to build NERIUM itself.

14.3 Competitors in the Marketplace and Infrastructure Space

Several agent marketplaces are emerging. Google Cloud AI Agent Marketplace offers verified agents for enterprise. ADP Marketplace offers AI agents for HR. Picsart launched an agent marketplace for creators. AI Agent Store functions as an agent directory. But all of these operate in specific domains or as extensions of existing platforms.

None combine an open marketplace, payment infrastructure, identity system, and interoperability protocol in one ecosystem. This is NERIUM's largest differentiator: not any one of the five pillars individually, but the combination and integration of all of them under one infrastructure thesis.

The recursive automation thesis (NERIUM built itself) is also a differentiator that no competitor can credibly replicate. The submission codebase contains the formal proof: the 54-plus specialist agents, the V1 through V6 handoff documents, the daily Ananke logs. The thesis is not marketing. It is a documented historical artifact.

15. Risks and Considerations

15.1 Quality of AI-Generated Code

AI models are powerful but not perfect. The code they produce can contain bugs, security vulnerabilities, and architectural weaknesses. NERIUM mitigates this risk through multi-agent review (agents check each other's work), dedicated QA and security agents, and transparency to the user about the limitations of AI-generated code. The risk remains real and must be communic-

CONFIDENTIAL / PROPRIETARY

ated clearly. The mitigation is structural rather than aspirational, and the QA agent in every Sekuri tier is the structural answer.

15.2 Scope Management

Users may describe ideas that are too large for AI agents to build reliably. "I want to build the next Tokopedia" is a real input the platform must handle. The platform needs robust mechanisms to help users narrow their idea scope to achievable milestones. The game metaphor helps here: limited resources in the game (limited workers, limited materials) naturally force users to prioritize. The Sekuri tier classification also acts as an explicit scope-management mechanism, by routing oversized prompts to the large tier and surfacing the cost projection upfront.

15.3 API Cost Uncertainty

For complex projects, AI compute costs can be significant. Users need clear visibility into projected costs before and during construction, with the ability to set budgets and receive alerts. A platform that fails to deliver cost transparency loses user trust. The Sekuri tier templates include explicit cost projections (USD 0.50 small, USD 2.00 medium, USD 8.00 large with multi-vendor) precisely to address this. The cost transparency is non-negotiable for the Builder pillar.

15.4 User Expectations

There is a risk that users expect NERIUM to produce the same quality as a professional development team. Clear communication about what the platform can and cannot do, and positioning it as an MVP builder rather than an enterprise solution at submission scope, is essential. The 2027 and 2029 capability targets in Section 17 are explicitly framed as forward-looking, and they are not implied to be currently shipping.

15.5 Cold Start Problem

Marketplace requires sufficient supply and demand to function. Registry requires a large enough base of agents to provide value. Protocol requires sufficient adoption to become a standard. Each pillar faces a different cold-start problem.

The mitigation strategy is sequencing. Builder is the first entry point and does not depend on network effects. Once the Builder user base is large enough, those users become the seed for Marketplace (as creators and buyers). Active Marketplace drives Banking. And so on. The launch order of pillars is planned strategically.

CONFIDENTIAL / PROPRIETARY

15.6 Regulation and Compliance

As governments worldwide begin regulating AI (EU AI Act, evolving United States regulations, and emerging frameworks elsewhere), platforms operating large-scale agent ecosystems must be prepared to comply with various regulatory frameworks. NERIUM must build compliance into the architecture from the start, not as an afterthought. The Registry pillar is the natural compliance surface, because verified capabilities and audit trails map directly to most emerging regulatory requirements.

15.7 Security and Trust

As a platform that handles software construction, financial transactions, and agent identity, NERIUM's attack surface is wide. Significant investment in security (technical and operational) is required from day one. The Registry pillar's security audit infrastructure is the first line of defense, and the Banking pillar's transaction security is the second.

15.8 Execution Complexity

Building any one of the five pillars is a large challenge. Building all five simultaneously is essentially infeasible at the scale a typical startup can resource. The phased launch strategy, with clear priorities on which pillar launches first, is mandatory. The hackathon submission is best understood as the prototype for the Builder hero pillar, with the four supporting pillars wired structurally but not yet at the same depth.

15.9 Solo Founder Risk

The hackathon submission was built by one founder using Claude Code as the throughput multiplier. Post-hackathon, scaling NERIUM to a real platform requires hiring or co-founding. The risk is that the recursive automation thesis ("NERIUM built itself") creates the impression that the platform can scale without human capital. This is not the claim. The claim is that NERIUM can scale further and faster with less human capital than conventional infrastructure. Founder hiring remains a genuine constraint and is part of the post-launch roadmap.

CONFIDENTIAL / PROPRIETARY

16. Success Metrics

NERIUM's success is measured across three primary dimensions, with specific metrics for each pillar.

16.1 Engagement and Adoption

Builder. How many users start a project. How many complete at least one phase. How many reach the Grand Opening (deployment). The completion rate is the most important metric, because it indicates whether gamification and AI orchestration are actually working in combination.

Marketplace. Number of registered agents. Monthly transaction volume. Supply-demand ratio. Buyer retention rate (what percentage return to buy again).

Banking. Total processing volume. Number of active agents processing transactions. Average transaction value.

Registry. Number of registered agents. Percentage of agents with "Verified" status. Frequency of trust score usage in purchase decisions.

Protocol. Number of supported AI models. Volume of cross-model communication processed. Adoption rate by third-party developers building on NERIUM Protocol directly.

16.2 Product Quality

The percentage of products deployed through Builder that are functional, stable, and usable by real end-users. This is measured through uptime monitoring, error rate tracking, and user feedback on deployed products. A Builder run that produces a non-functional output is a failed run, regardless of whether the agents completed their tasks. The metric is end-user functional, not internal task-completion.

16.3 Economic Viability

Average revenue per user. AI compute cost per completed project. User lifetime value. Unit economics for each pillar. The platform must deliver enough value to justify its costs at every pillar. The flywheel effect described in Section 11.7 is real only if the unit economics at each pillar are individually defensible.

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

16.4 Submission-Window Metrics

Specific to the hackathon submission, three additional metrics matter:

1. **Demo completion rate.** The percentage of judges and reviewers who complete the full theatrical demo flow. Target: above 80 percent.
2. **BYOK validation rate.** The percentage of technical reviewers who supply their own API key and execute a live Builder run. Target: above 20 percent.
3. **Repository engagement.** Stars, forks, and substantive issues on `github.com/Finerium/nerium` within seven days of submission. Target: above 100 stars in week one.

These metrics are submission-specific and roll into the broader engagement metrics post-launch.

---

17. Future Vision

NERIUM begins as a platform for individuals and small businesses, but the long-term potential extends substantially further. The trajectory is articulated explicitly in the Session 2 voiceover, and the milestones are dated.

17.1 The 2027 Milestone: Idea to App Store

By 2027, NERIUM Builder ships your idea straight to the App Store as a production-grade app, ready for hundreds of thousands of downloads on launch day. This is not generation of a prototype that you then refine for production. This is the production deployment itself, in a single conversation.

The 2027 milestone requires the multi-vendor runtime to be live across all eight vendors. It requires the Marketplace to have populated supply across multiple verticals. It requires Banking to be operating live with Stripe Atlas activated. It requires Registry to have a critical mass of verified agents. It requires Protocol to have hardened across the eight vendors and any new vendors that have entered the market in the intervening period. Each of these requirements has a path. The hackathon submission is the proof that the path is shippable.

CONFIDENTIAL / PROPRIETARY

17.2 The 2029 Vision: Civilization-Scale Infrastructure

By 2029, software development is no longer a profession bottleneck. Every business runs on agents. Every agent runs on NERIUM. Ideas, not codebases, create wealth.

This is the civilization-scale framing, and it deserves to be unpacked. "Software development is no longer a profession bottleneck" means that the rate-limiting step of the modern economy (the supply of skilled software engineers) is removed. "Every business runs on agents" means that the marginal cost of operating a business with custom software collapses to near zero. "Every agent runs on NERIUM" means that the infrastructure layer wins, in the way that AWS won compute and Stripe won payments. "Ideas, not codebases, create wealth" means that the value-capturing input shifts from technical execution to conceptual design.

The implications are vast. Indonesian high school students with good ideas can ship products that compete globally. Restaurant owners can build operational software that fits their specific workflow at the cost of an afternoon's thinking. Researchers can prototype experimental tooling without engineering teams. The marginal cost of bringing a new digital service into existence approaches zero, and the surplus that this generates accrues to the people with the ideas rather than the people with the technical skills.

This is not a prediction made lightly. It is a thesis grounded in the trajectory of AI capability, the obvious gap in infrastructure, and the recursive automation proof that the hackathon submission represents. NERIUM is the bet that this trajectory is correct and that the infrastructure to ride it should be built now.

17.3 Multiplayer Mode

Multiple users collaborating on a single building, each contributing to different aspects of the product. One user handles business logic, another handles design, AI agents work for both. This opens NERIUM to teams and organizations, not just individuals. The multiplayer feature is preserved from PRD v1.0 but reframed under the civilization-infrastructure thesis: multiplayer is what makes NERIUM the substrate for distributed teams, and distributed teams are what make NERIUM the substrate for global product creation.

17.4 Building District

Users can see a "city" of all buildings under construction on the platform. This creates a sense of community, allows for discovery of interesting projects, and lets users learn from each other's approaches. The Building District is not just visualization. It is a network effect surface, because the proximity of buildings in the district maps to discoverability. Two adjacent buildings (two adjacent projects) share visibility, and shared visibility is the core mechanic of community.

CONFIDENTIAL & PROPRIETARY

17.5 Maintenance Mode

Once a building is complete, it is not abandoned. The platform continues to monitor it, handles updates, fixes emerging issues, and scales infrastructure as needed, all visualized as building maintenance. Maintenance Mode is the substrate for ongoing operations, and it locks NERIUM into the user's ongoing infrastructure spend. A building is not a one-time purchase. A building is a relationship.

17.6 Building Upgrades

Users can return to their buildings at any time to add new floors (features), renovate existing ones (refactoring), or expand the building overall (scaling). The upgrade surface is what turns a Builder output into a long-term product, and the long-term product is what generates Banking revenue at scale.

17.7 Agentic Commerce

As the agent economy matures, agents will begin to act as autonomous buyers, reshaping procurement and discovery models in e-commerce. NERIUM Banking and Registry are positioned ideally to handle this era. When one agent purchases services from another agent, the transaction needs to clear, the trust needs to verify, and the routing needs to optimize. All three are NERIUM functions.

17.8 Personal Agent Ecosystem

Each individual will eventually own a "fleet" of personal agents handling various aspects of their lives: finance, health, productivity, communication. NERIUM becomes the platform where this fleet is managed, updated, and (where the user chooses) monetized. The personal agent ecosystem is the consumer-facing endgame of the entire platform.

17.9 Industry Standard

If NERIUM Protocol and Registry are widely adopted, they may become de facto industry standards for the agent economy, similar to how HTTP became the web standard and SWIFT became the international banking standard. Standards win when they become defaults, and the path to default for NERIUM Protocol runs through Builder, Marketplace, and Banking. Every Builder user encounters Protocol through the multi-vendor selection modal. Every Marketplace transaction touches Registry for trust verification. Every Banking transaction touches Protocol for routing. The pillars compound into a standard whether any individual user thinks of it that way or not.

CLASSIFIED
CONFIDENTIAL / PROPRIETARY

17.10 The Founder's Note

A final thought, written directly. NERIUM was built by one Indonesian student during a hackathon, using an AI workflow that he had been refining across his projects for the past year. That workflow includes a 47-agent IDX investment AI blueprint that mapped every step of an agentic financial pipeline, a parallel multi-agent academic project building a medical-themed premium frontend showcase portfolio, a multi-month XAU/USD algorithmic trading system assembled locally across 3,808 passing tests, and a long string of smaller experiments in multi-agent orchestration. Each of those projects taught the founder one specific lesson about how agent pipelines fail at scale. NERIUM is the platform that addresses every failure mode the founder has lived through.

This is not a corporate product. This is not a generic startup pitch. This is one founder's attempt to build the infrastructure that he wishes had existed when he was running 30-step manual handoff pipelines to ship MVPs in his dorm room. The thesis is sharp because it is lived. The product is shippable because the founder has shipped many smaller versions of it already. The civilization-scale ambition is honest because the founder believes, after a year of doing this work, that the trajectory is real and that the infrastructure to ride it should be built now.

The hackathon submission is the proof. The next version is the work.

Closing

This document represents the conceptual foundation of NERIUM as it stands at hackathon submission, April 2026. It is intended to communicate vision, current shipped state, and forward trajectory. Technical specifications, technology stack decisions, detailed engineering plans, and go-to-market strategies live in adjacent documents in the orchestration log.

NERIUM is not just another AI platform. NERIUM is the bet that the AI agent economy will become one of the largest economic layers in the next decade, and that whoever controls the ecosystem infrastructure will occupy a position comparable to AWS, Visa, or Google in their respective eras.

The hackathon submission is the proof of concept. The post-launch roadmap is where the bet plays out.

CONFIDENTIAL / PROPRIETARY

CLASSIFIED

NERIUM PRD v2.0, April 2026. Authored by Ghaisan Khoirul Badruzaman.

Open source under MIT license at `github.com/Finerium/nerium`.

Built with Opus 4.7. Built by 54-plus specialist Claude Code agents. Built once, manually, the way NERIUM Builder will build everything else.

CLASSIFIED

CONFIDENTIAL / PROPRIETARY